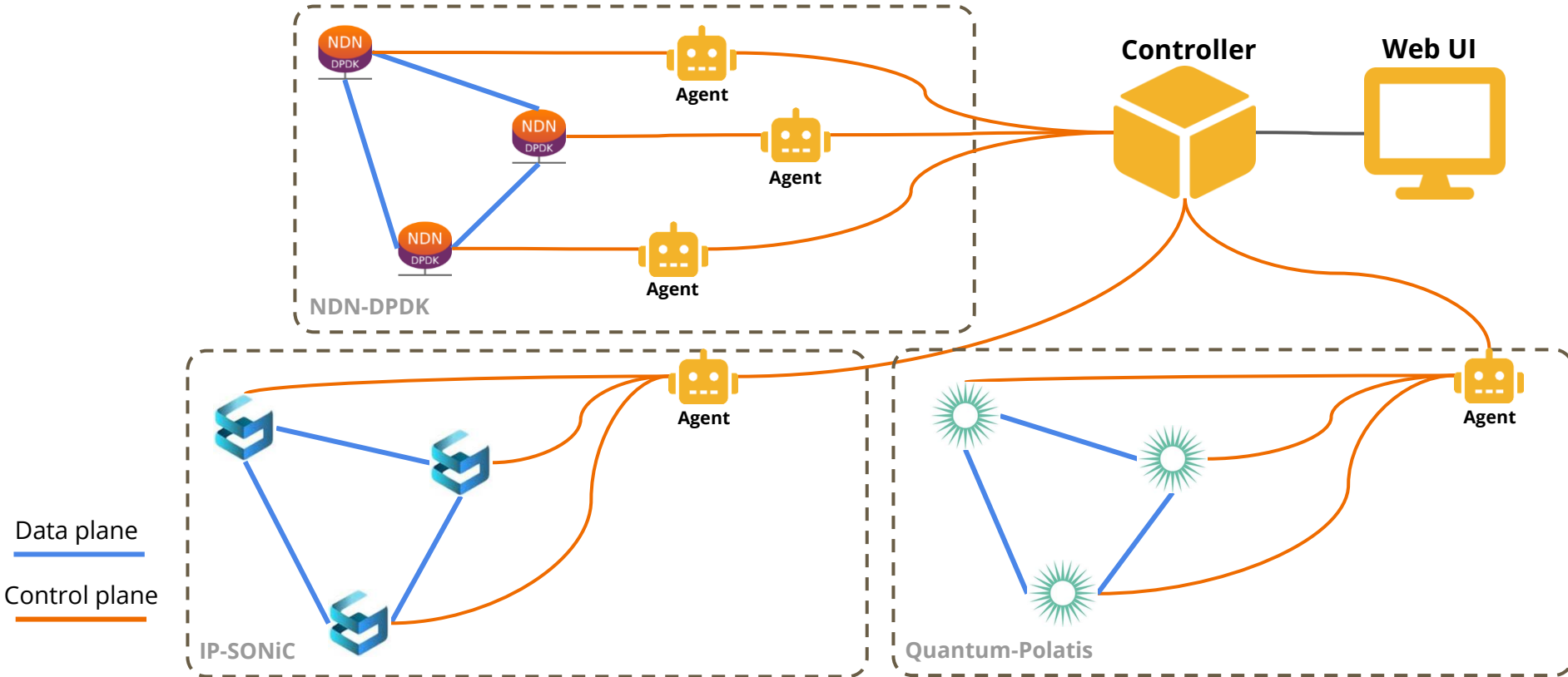


Multiverse: Designing a Network Management System with NDN

A. Abane, D. Pesavento, J. Shi, M. Merzouki, L. Benmohamed, A. Battou
Communications Technology Laboratory



Multiverse Architecture



Current Management Use Cases

Network technology	State/config collection	Target platform	Control plane	Stack complexity	Scale	Telemetry
IP (SONiC)	Full	Limited extensibility	IP, VLAN, iBGP, ACL	High	Medium-Large	Shared Capability platform
NDN (NDN-DPDK)	Partial	Open	Faces, Routes	Medium	Medium	
Quantum (Optical switches)	Minimal	Closed	Cross-connects	Low	Small-Medium	

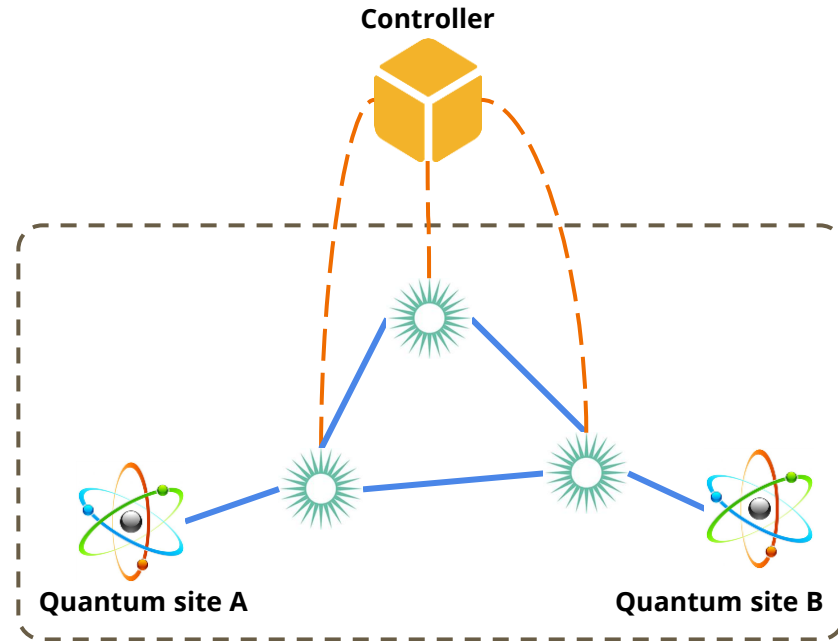
Research Areas

Network Intelligence	NDN-over-Ethernet, IP-only, Cross-connect Query network state, error prevention, error detection	Digital Twin service based on a single network model
Network Management APIs	Not all network engineers/devs are familiar with NDN Quick prototyping (IP) which can be easily switched to NDN later Use full-managed environments and focus on innovation Lots of management APIs to design and maintain	Leverage NDN security/naming Pub/sub abstraction

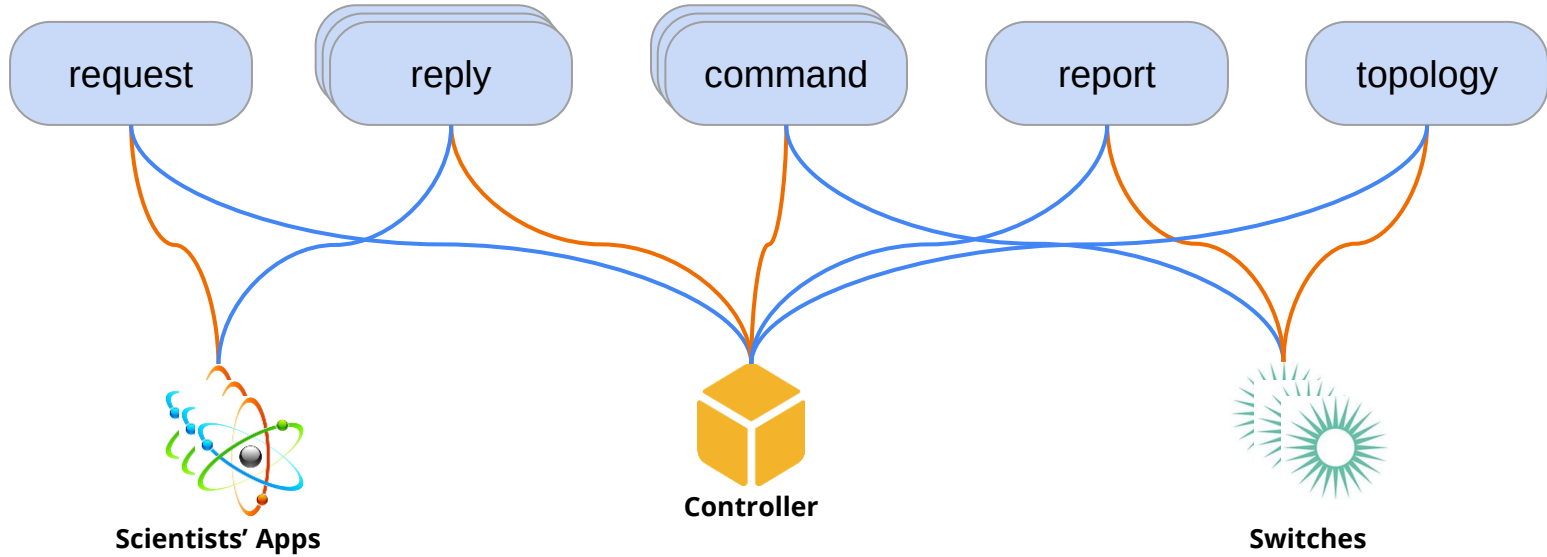
Quantum Networks Management

Optical network path scheduling

1. Application requests path (between two quantum sites) to controller.
2. Controller computes path and asks optical switches to setup cross-connects.
3. Application renews the path if it needs more time.
4. When teardown time arrives, controller asks optical switches to teardown cross-connects.
5. Both quantum sites must approve the optical connection.



Path Provisioning...



[Publish](#)

[Subscribe](#)

...with Pub/sub Abstraction

Optical Path Scheduling Service 1.0.0

This service is in charge of provisioning optical paths for quantum experiments.

Operations

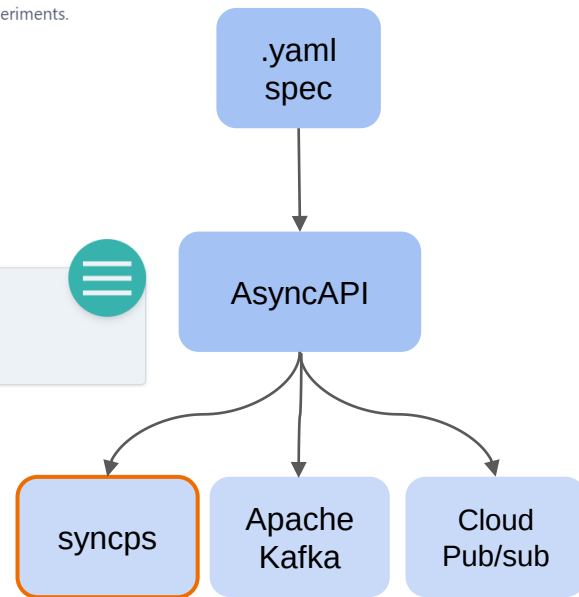
SUB /qnet/request

Accepts the following message:

Request

Payload >

Object



```
asyncapi: '2.1.0'
info:
  title: Optical Path Scheduling Service
  version: 1.0.0
  description: This service is in charge of provisioning
optical paths for quantum experiments.
channels:
  /qnet/request:
    subscribe:
      message:
        $ref: '#/components/messages/Request'
components:
  messages:
    Request:
      payload:
        type: object
        properties:
          ports:
            type: array
            items:
              type: string
            description: Exactly 2 ports
```

Open Questions - Future Work

- Content encryption/decryption in Pub/sub protocols (NDN)
 - Transparent to applications [ref. syncps]
 - Without altering Pub/sub primitives
- Describe Pub/sub APIs AND security
 - Topics, message formats, data trust, access control
 - Machine and human readable specs, security review/approval, team collaboration
 - Leverage VerSec approach, AsyncAPI
- API code/docs generation for NDN-based Pub/sub protocols
 - Automate some tasks as adopted in modern application development
 - Allow to start with NDN from familiar concepts (Pub/sub, API-first)
 - Migration from IP-based Pub/sub to NDN
- Extend Digital-Twin to NDN
 - What can be misconfigured? How to detect it?
 - More critical when routing protocol will be used (NDN-DPDK)

Thank you!